A 65.1 TOPS/W Digital CIM Processor for Ultra-Low-Bit Transformers with Multiplexer-based Adder and Scaling Factor-based Reordering

Nayeong Lee, Sangyeob Kim, Seongyon Hong, Jiwon Choi, and Hoi-Jun Yoo
School of Electrical Engineering, Korea Advanced Institute of Science and Technology (KAIST)
Daejeon, Republic of Korea
Email: ny.lee@kaist.ac.kr

2025 IEEE International Symposium on Circuits and Systems (ISCAS) | 979-8-3503-5683-0/25/$31.00 ©2025 IEEE | DOI: 10.1109/ISCAS56072.2025.11043703

***Abstract*— As large-language-model (LLM) continues to expand in parameter size and improve performance, challenges related to latency and energy efficiency become increasingly significant. While low-bit weight quantization was proposed to alleviate the external memory access bottleneck, it remains insufficient to reduce overall energy consumption. In this paper, we use binarized LLM and design a novel Computing-in-Memory (CIM) processor to reduce both computation and memory energy. We propose three key techniques: 1) Multiplexer-based adder is proposed to replace first stage adders of adder tree with the simple multiplexers and inverter, reducing adder tree energy consumption by 58%, 2) Scaling factor-based reordering is proposed to reduce adder tree bit-width, resulting in an additional 28.6% energy reduction, 3) Cluster-level broadcasting and bit-spatial mapping is proposed to reduce internal memory access and increase bit-scalability. As a result, we can reduce the overall energy consumption by 69.7%, offering a more efficient solution for LLM inference tasks.**

Keywords*— *Computing-in-Memory (CIM), Large-language-Model, Transformer, ShiftAddLLM

I. Introduction

Recently, as the parameter size of large language models (LLMs) has grown, performance has continuously improved. However, this increase caused significant challenges in latency and energy efficiency. To address the bottleneck due to external memory access (EMA), various quantization methods have been proposed to compress LLM parameters. In particular, low-bit weight quantization methods [1-3] that reduce precision to below 4 bits have been actively researched. ShiftAddLLM [4], as shown in Fig. 1 (a), further reduced the bit precision required per parameter to 1-3 bit by decomposing the original weight into a combination of a scaling factor (in the form of 2^n) and binary weights. The binary weights replace high-cost multiplications with simple additions. Therefore, binarized LLM using the method from [4] can effectively reduce energy consumption while maintaining better perplexity compared to previous works [1-3].

Binarized LLM requires a large number of parallel additions. This characteristic is well-suited for digital Computing-in-Memory (CIM) processors because CIM can efficiently handle highly parallel operations directly within memory [5]. However, previous digital CIMs adopted a bit-serial operation method, where each bit of an operand is delivered one by one to the word line in each cycle to perform multiplication. When using binary weights, multiplication can be replaced by addition, eliminating the need for serial processing. Furthermore, by excluding the additional logic required for multiplication, CIM specialized for binarized LLM can reduce both latency and energy consumption compared to previous CIM processors.

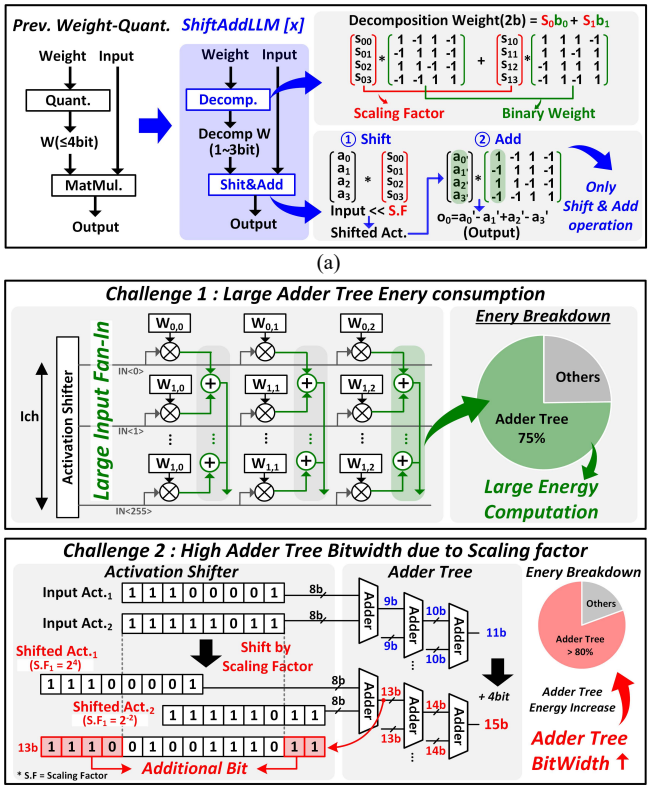


Fig. 1. Challenges of computing-in-memory processor for ShiftAddLLM with binarized weight.

However, there are significant challenges in implementing an energy-efficient CIM for binarized LLM, as shown in Fig. 1 (b). First, digital CIM requires large computational energy consumption due to the adder tree. At the macro-level, a large amount of addition is performed in the bit-line direction for input channel-wise accumulation within a single cycle, making the adder tree energy a major contributor to overall energy consumption. An analysis of previous digital CIM processors shows that the adder tree accounts for more than 75% of total energy consumption. Even if CIM is designed to process binarized LLM with ShiftAddLLM, a high degree of parallel accumulation still occurs along the bit line direction, resulting in the adder tree consuming a large portion of power. Second, in ShiftAddLLM, each activation is shifted by a different scaling factor before being multiplied by the weight, which leads to varying bit widths for the shifted activation. This increases the bit-width of the adder tree in digital CIM. Thus, the adder tree is a critical source of power drain, and addressing this issue is essential to improving the energy efficiency of the entire system.

In this paper, we propose a new CIM architecture to reduce the energy consumption of the adder tree and maximize the

Authorized licensed use limited to: University of Electronic Science and Tech of China. Downloaded on September 29,2026 at 12:17:37 UTC from IEEE Xplore. Restrictions apply.

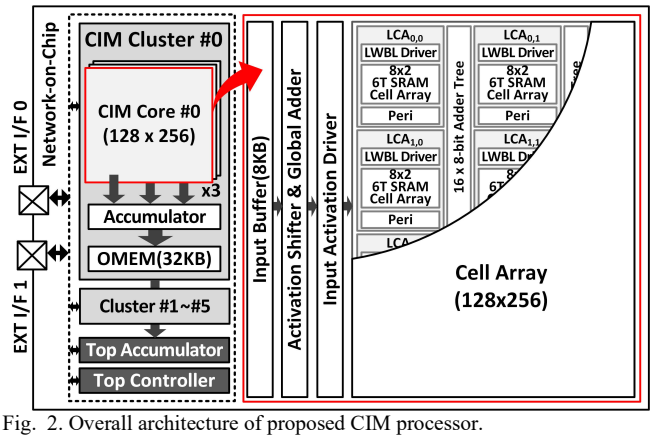


Fig. 2. Overall architecture of proposed CIM processor.

energy efficiency of the entire system. The proposed processor reduces energy consumption through the following three key techniques.

1) By utilizing the characteristic of binary weights being represented as *'-1'* and *'+1'*, we propose multiplexer-based adder to replace the first-stage adders with two multiplexers and an inverter. As a result, CIM Macro energy efficiency increases by 58%.

2) To reduce the large bit-width of the adder tree caused by scaling factors, we propose a scaling factor-based reordering that allows operations using similar scaling factors to be processed simultaneously in CIM. As a result, when combined with the multiplexer-based adder, the energy consumption of the macro is reduced by 69.7%.

3) Cluster-level broadcasting is proposed to reduce internal input and output memory access, resulting in a 33.3% reduction in internal memory access. Additionally, bit-spatial mapping improved bit scalability for multi-bit weight ShiftAddLLM operations.

II. Proposed Computing-In-Memory Processor

A. Overall Architecture

Fig. 2 shows the overall architecture of the proposed CIM processor. The proposed processor consists of 6 clusters, top controller, top accumulator, a Network-on-Chip (NoC), and 2 external interfaces. Each CIM cluster comprises 3 CIM cores, an accumulator, and 32KB OMEM.

Each CIM core consists of a 128×256 CIM cell array, an 8KB input buffer, an activation shifter, a global adder, and an input activation driver. Each local cell array (LCA) consists of an 8×2 SRAM cell array. The peripheral circuit of LCA (LCA peri) consists of two multiplexers and one inverter instead of an adder to perform the first-stage addition of the adder tree. The remaining stages of the adder tree are integrated along the bit-line direction, performing accumulation across the input channels. The input buffer stores the activations, which are multiplied by the scaling factor in the form of 2^n. These activations are then transferred to the activation shifter and global adder to generate the possible computation results based on the binary weights. Depending on the combination of binary weights from two adjacent cells in the LCA, the appropriate computation result is selected via the multiplexer in the LCA peri to perform the first-stage of the adder tree operation. The selected result passes through the remaining stages of the adder tree, accumulating the partial sums in the

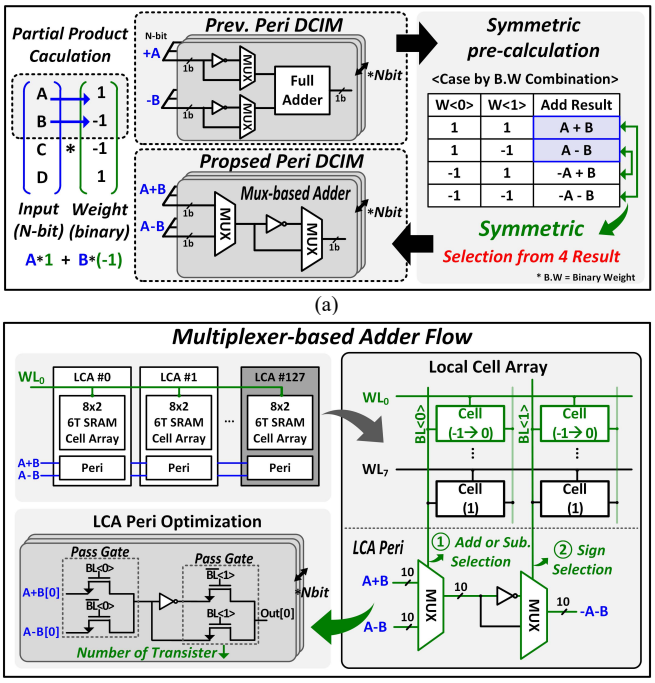


Fig. 3. Multiplexer-based adder for binarized LLM.

input channel direction. Finally, the accumulation results are transferred to the accumulator for the final output calculation.

B. Multiplexer-based Adder for Binarized LLM

In Digital CIM, the *N*-bit data is shared in the word line (WL) direction, allowing simultaneous accumulation across multiple columns, which results in over 75% of total energy consumption being attributed to the adder tree. The first-stage of the adder tree comsumes ~50% of total energy consumption within the entire adder tree, highlighting the need for a method to reduce the energy consumption at this stage.

The binarized LLM multiplies the shifted activations by either *"-1"* or *"+1"*, and they must be accumulated in the input channel direction by the adder tree. Depending on the combination of binary weights from the two input channels, the first stage of the adder tree calculates one of four possible partial sum, as shown in Fig. 3(a). By pre-calculating the four outcomes and selecting one based on the weights, the first-stage adders can be eliminated. Additionally, in transformer operations for LLM, activations are reused across multiple output channels or batches, allowing the pre-calculated results to be reused spatially across multiple columns of the CIM macro or temporally across several cycles. Furthermore, these four outcomes exhibit symmetry: inverting *"A+B"* yields *"-A-B"*, and inverting *"A-B"* yields *"-A+B"*. Thus, if only two computation results of the two operands are pre-calculated, numerous adders involving these operands can be replaced with simple multiplexers and inverter.

The proposed multiplexer-based adder uses two 2-to-1 multiplexers and one inverter. When the two pre-calculated values (*"A+B"*, *"A-B"*) are provided as inputs, it simply generates one of the four computation results based on the binary weight combination stored in two adjacent cells. Each LCA consists of an 8×2 SRAM cell array, and when WL is activated within the LCA, the binary weights stored in the two cells are read through *BL<0>* and *BL<1>*. The value read from *BL<0>* is used as the selection signal for the first

Authorized licensed use limited to: University of Electronic Science and Tech of China. Downloaded on September 29,2026 at 12:17:37 UTC from IEEE Xplore. Restrictions apply.

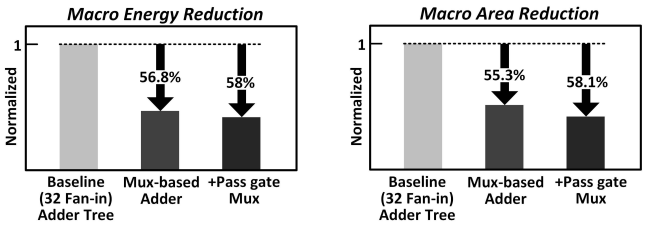


Fig. 4. Performance of proposed Multiplexer-based Adder.

multiplexer, which selects one of the two pre-calculated values. The second multiplexer and inverter then generate the remaining two symmetric computation results (*"-A+B", "-A-B"*) based on the weight combination. The value read from *BL<1>* serves as the selection signal for the second multiplexer, determining whether to invert the output from the first multiplexer or to pass it through unchanged. Fig. 3(b) shows an example of processing computations for two input channels (*A, B*). If the read values are *BL<0>=0* and *BL<1>=0*, both weights are *"-1"*, so *"-A-B"* should be the final output. First, *BL<0>=0* is used as the selection signal for the first multiplexer, which outputs *"A+B"*. Then, since *BL<1>=0*, *"A+B"* passes through the inverter, resulting in *"-A-B"* being output from the second multiplexer. Through this, the final result for the two operands is accurately calculated. In addition, by implementing the 2-to-1 multiplexer using pass gates, both area and energy consumption are minimized. Since the pass gates require only two transistors per bit, they are more efficient in terms of energy and area compared to conventional digital multiplexers implemented with the static logic like AND/OR in conventional digital hardware. Consequently, the area of the macro is reduced by 58.1%.

Fig. 4 shows the energy consumption achieved through the proposed multiplexer-based adder. By using multiplexer operations instead of adder operations, we achieve ~56.8% energy reduction in the entire CIM macro compared to the baseline implementation using a conventional adder tree with 32 Fan-in. Furthermore, implementing the proposed adder tree with multiplexers using pass gates results in up to 58% energy reduction compared to the baseline.

C. Scaling Factor-based Input Channel Reordering

In ShiftAddLLM, the parameter is decomposed into a combination of a scaling factor (in the form of 2^n) and binary weights, expressed as ($S.F._0*W_0 + S.F._1*W_1$). The scaling factor takes different values in the input channel direction and is shared across the output channels. The scaling factor and activation are first processed in the activation shifter to generate shifted activations, which are then computed with the binary weights. Since the scaling factor varies for each input channel, the range of data that needs to be represented differs for each channel. The proposed CIM performs accumulation across a fixed number of input channels in the column direction. Specifically, 8 WLs in each LCA are sequentially activated, performing addition for two input channels per cycle, while the adder tree receives partial sums from LCAs and performs accumulation. By changing the computation order so that each LCA processes only input channels with similar scaling factor values in each cycle, the unnecessary bit-width in the addition process can be reduced.

To address this, we propose scaling factor-based input channel reordering. Since the scaling factors are pre-trained values, we can rearrange the input channels in advance. The

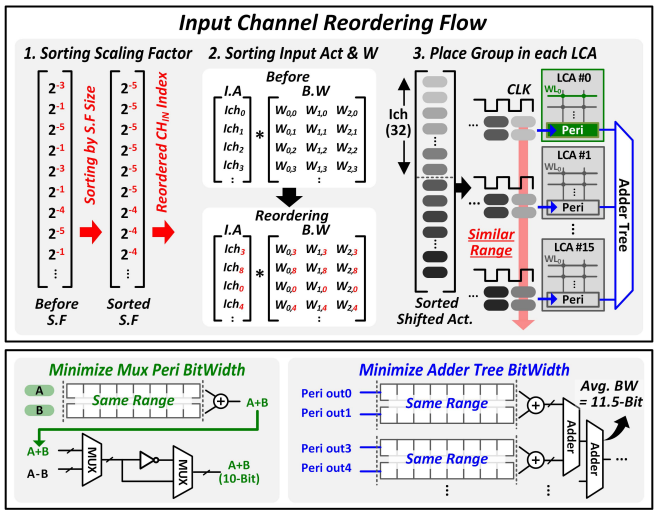


Fig. 5. Scaling factor-based input channel reordering.

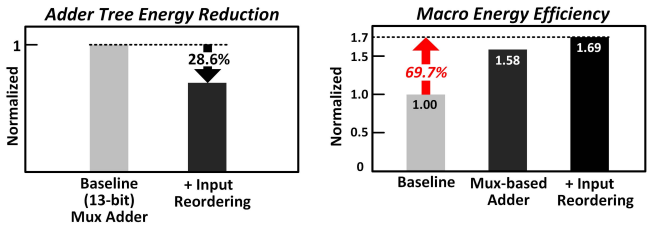


Fig. 6. Performance of proposed input channel reordering.

reordering consists of two stages: inter-reordering and intra-reordering. Inter-reordering ensures that the input channels with similar scaling factors are assigned within the same CIM macro. For this, the input channels are reordered according to the magnitude of their scaling factors, as shown in Fig. 5. Since each macro processes 128 input channels, the input channels reordered by magnitude are divided into groups of 128. These groups are then assigned to each macro in order, starting with the group with the smallest scaling factors. Intra-reordering performs a detailed rearrangement to minimize the bit-width of the adders in accordance with the characteristics of LCAs. There are 16 LCAs per column in the CIM macro, and each LCA processes the addition of two input channels for each cycle and transfers the result to the adder tree. In order to make the bit-width required for LCA operation results of each cycle similar, two input channels are allocated to the LCAs in order of the scaling factor magnitude. After all LCAs are allocated, two input channels are allocated again from the first LCA in order of the scaling factor magnitude, and this process is repeated until all input channels are allocated. This approach ensures that the bit-width of the values output by the LCAs in one cycle becomes nearly identical or similar, minimizing the adder tree bit-width in the CIM macro.

Fig. 6 shows the energy reduction by the scaling factor-based input channel reordering. Without the reordering, the multiplexers in each LCA peri require a bit-width of up to 13 bits due to the varying scaling factors of the input channels. Moreover, adders for the remaining stages require a bit-width of 13 to 16 bits. After applying input channel reordering, the multiplexers require only 10 bits, and the remaining stages can be implemented with adders of 10 to 13 bits. When input channel reordering is applied, the energy consumption of the adder tree is additionally reduced by 28.6%. As a result, applying this with the multiplexer-based adder improved the energy efficiency of the entire macro by up to 69.7%.

Authorized licensed use limited to: University of Electronic Science and Tech of China. Downloaded on September 29,2026 at 12:17:37 UTC from IEEE Xplore. Restrictions apply.

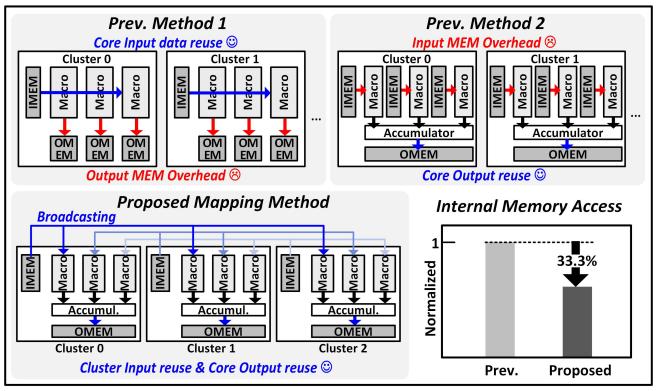


Fig. 7. Cluster-level broadcasting for internal memory access reduction.

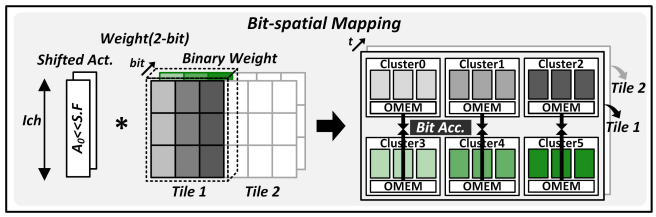
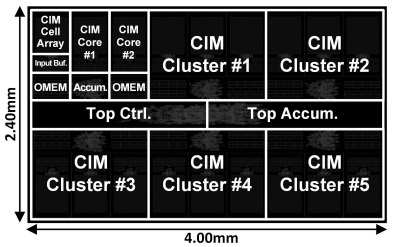


Fig. 8. Bit-spatial mapping method for high bit-scalability.

D. *Bit-spatial Mapping with Cluster-level broadcasting.*

Weight stationary operations in multi-core architectures have traditionally been mapped using two methods. As shown in Fig. 7, method 1 allows macros in the cluster to reuse input data from the same input channel by accessing a shared input memory. However, since each macro processes a different output channel, each macro accesses its dedicated output memory, increasing internal memory access. Conversely, method 2 reduces output memory access by accumulating partial sums from all macros for the same output channel and only storing the final result in the output memory. However, since each macro requires data from a different input channel, this leads to increased input memory overhead. The proposed method follows the same approach as method 2, accumulating outputs through an accumulator and avoiding output memory access overhead. In addition, it solves the issue of requiring data from different input channels for each macro by enabling input broadcasting between clusters. Each cluster stores different input channel data, and the required input data is broadcasted to other clusters, reducing unnecessary input memory access. This approach reduces internal memory access by 33.3% compared to method 1 and 2.

Moreover, ShiftAddLLM [4] uses multiple pairs of scaling factors and binary weights for complex and accurate tasks, and the proposed method efficiently handles cases that require multi-bit precision per parameter. In multi-bit precision, the results of each binary weight operation must be repetitively stored and retrieved to generate the final output, leading to frequent output memory access. To address this, a spatial mapping method is used to minimize internal memory access. The proposed mapping method allocates each decomposed bit across macros. Fig. 8 illustrates the mapping method when the weight is decomposed into 2 bits. The 18 macros are divided into two groups of 9, each assigned to a different bit. Since each cluster generates 128 output channels and 3 clusters are assigned per bit, they generate partial sums of 384 output channels. Finally, the outputs from the two clusters for the two bits of the same output channel are added for bit-wise accumulation. This mapping accumulates operations across input channels and bit directions simultaneously, improving bit scalability while reducing internal memory access.



OPT-125M	Bits	PPL	OPT-125M	Bits	PPL
FP16	16	27.65	FP16	16	27.65
OPTQ[1]	3	53.85	OPTQ[1]	2	2467.50
LUT-GEMM[2]	3	60.00	LUT-GEMM[2]	2	4844.32
AWQ[3]	3	54.75	AWQ[3]	2	3514.61
Ours. With [4]	3	32.19	Ours. With [4]	2	52.34

* Benchmark Test: WikiText2

		This Work
Technology[nm]		28
Die Area (mm^2)	Core	0.122
	Total	9.6
Bit Precision	Act	INT8
	Weight	INT 1~3
Supply Voltage(V)		1
Frequency(MHz)		250
Power(W)		0.058
Macro Energy Efficiency (TOPS/W)		83.5 (INT1) 41.7 (INT2)
Overall Energy Efficiency (TOPS/W)		65.1 (INT1) 32.5 (INT2)

* Benchmark Test: OPT-125M, WikiText2

Fig. 9. Layout Photograph and Performance Summary

TABLE 1. COMPARISON TABLE

	ISSCC'22 [6]	ISSCC'23 [7]	ISSCC'23 [8]	JSSCC'24 [9]	This Work
Technology[nm]	28nm	28nm	28nm	28nm	28nm
Supply Voltage(V)	0.6-1.0	0.6-1.0	0.64-1.03	0.6-1.0	1.0
Frequency(MHz)	80-240	85-275	20-320	80-275	250
Area(mm^2)	6.83	14.36	3.93	6.98	9.6
CIM Size [KB]	24	128	144	64	74
Bit Precision: Act	INT8/16	INT8/16	INT8	INT8/16	INT8
Bit Precision: Weight					INT 1~3
Operation Type	Mult.	Mult.	Mult.	Mult.	Shift & Add
Energy Efficiency* (TOPS/W)	12.5 (INT8) 3.1 (INT16)	48.4 (INT8) 12.1 (INT16)	2.52	15.7 (INT16)	65.1 (INT1) 32.5 (INT2)

* Measured with no sparsity condition

III. Implementation Results

Fig. 9 shows the layout of the proposed CIM processor. The processor is implemented using 28nm Samsung CMOS technology, with a total die area of 9.6mm^2, and each CIM macro occupies an area of 0.122 mm^2. The processor operates at clock frequency of 250 MHz with supply voltage of 1.0 V. The proposed key techniques reduce energy consumption by 69.7%, achieving high energy efficiency across the entire system. Despite this, the accuracy drop on the WikiText2 dataset is less than 1%, maintaining a low perplexity of 32.19. Table 1 shows the comparison table with the previous transformer CIM processors. Without any sparsity conditions, the proposed CIM processor achieves 34.5% higher energy efficiency compared to the previous state-of-the-art [7]. As a result, the macro achieves up to 83.5 TOPS/W, and the overall achieves up to 65.1 TOPS/W in energy efficiency.

IV. Conclusion

This paper proposes a CIM processor that can perform energy-efficient LLM with binary weights. By utilizing a new LCA peripheral circuit based on multiplexers and inverters, power-consuming adders in the first stage of the adder tree are eliminated. This reduces the energy consumption of the CIM macro by 58%. In addition, scaling factor-based input channel reordering allows each CIM macro to handle groups of data with a similar range, reducing the bit-width of the macro's multiplexer-based adder and adder tree. As a result, the adder tree's energy consumption decreases by 28.6%, and applying both multiplexer-based adder and reordering reduces the energy consumption of the macro by up to 69.7% Finally, cluster-level input broadcasting and output accumulation between macros within the cluster reduce internal memory access by 33.3%. Moreover, by adopting a bit-spatial method, the proposed CIM achieves high bit scalability for multi-bit weight operations. In conclusion, the proposed CIM processor with binarized LLM achieves high energy efficiency, achieving 65.1 TOPS/W across the entire system.

Authorized licensed use limited to: University of Electronic Science and Tech of China. Downloaded on September 29,2026 at 12:17:37 UTC from IEEE Xplore. Restrictions apply.

V. ACKNOWLEDGEMENT

This work was supported by Institute of Information & communications Technology Planning & Evaluation (IITP) under the Graduate School of Artificial Intelligence Semiconductor(IITP-2025-RS-2023-00256472) grant funded by the Korea government(MSIT). The EDA tool was supported by the IC Design Education Center(IDEC), Korea.

REFERENCES

[1] Elias Frantar, Saleh Ashkboos, Torsten Hoefler, et al. OPTQ: Accurate Quantization for Generative Pre-trained Transformers. In ICLR, 2022.

[2] Gunho Park, Baeseong Park, Minsub Kim, et al. LUT-GEMM: Quantized Matrix Multiplication based on LUTs for Efficient Inference in Large-Scale Generative Language Models. arXiv preprint arXiv:2206.09557, 2022.

[3] Ji Lin, Jiaming Tang, Haotian Tang, et al. AWQ: Activation-aware Weight Quantization for LLM Compression and Acceleration. arXiv preprint arXiv:2306.00978, 2023.

[4] Haoran You, et al. ShiftAddLLM: Accelerating Pretrained LLMs via Post-Training Multiplication-Less Reparameterization. arXiv:2406.05981v2, 2024.

[5] Christopher Wolters et al. Memory Is All You Need: An Overview of Compute-in-Memory Architectures for Accelerating Large Language Model Inference arXiv:2406.0841, 2024.

[6] F. Tu et al., "A 28nm 15.59μJ/Token Full-Digital Bitline-Transpose CIM-Based Sparse Transformer Accelerator with Pipeline/Parallel Reconfigurable Modes," 2022 IEEE International Solid- State Circuits Conference (ISSCC), San Francisco, CA, USA, 2022.

[7] F. Tu et al., "16.1 MuITCIM: A 28nm 2.24μJ /Token Attention-Token-Bit Hybrid Sparse Digital CIM-Based Accelerator for Multimodal Transformers," 2023 IEEE International Solid-State Circuits Conference (ISSCC), San Francisco, CA, USA, 2023.

[8] Liu. Shi et al., "16.2 A 28nm 53.8TOPS/W 8b Sparse Transformer Accelerator with In-Memory Butterfly Zero Skipper for Unstructured-Pruned NN and CIM-Based Local-Attention-Reusable Engine, " 2023 IEEE International Solid-State Circuits Conference (ISSCC), San Francisco, CA, USA, 2023.

[9] Ruiqi Guo et al., "CIMFormer: A Systolic CIM-Array-Based Transformer Accelerator With Token-Pruning-Aware Attention Reformulating and Principal Possibility Gathering, " 2024 IEEE Journal of Solid-State Circuits (JSSCC), 2024.

Authorized licensed use limited to: University of Electronic Science and Tech of China. Downloaded on September 29,2026 at 12:17:37 UTC from IEEE Xplore. Restrictions apply.